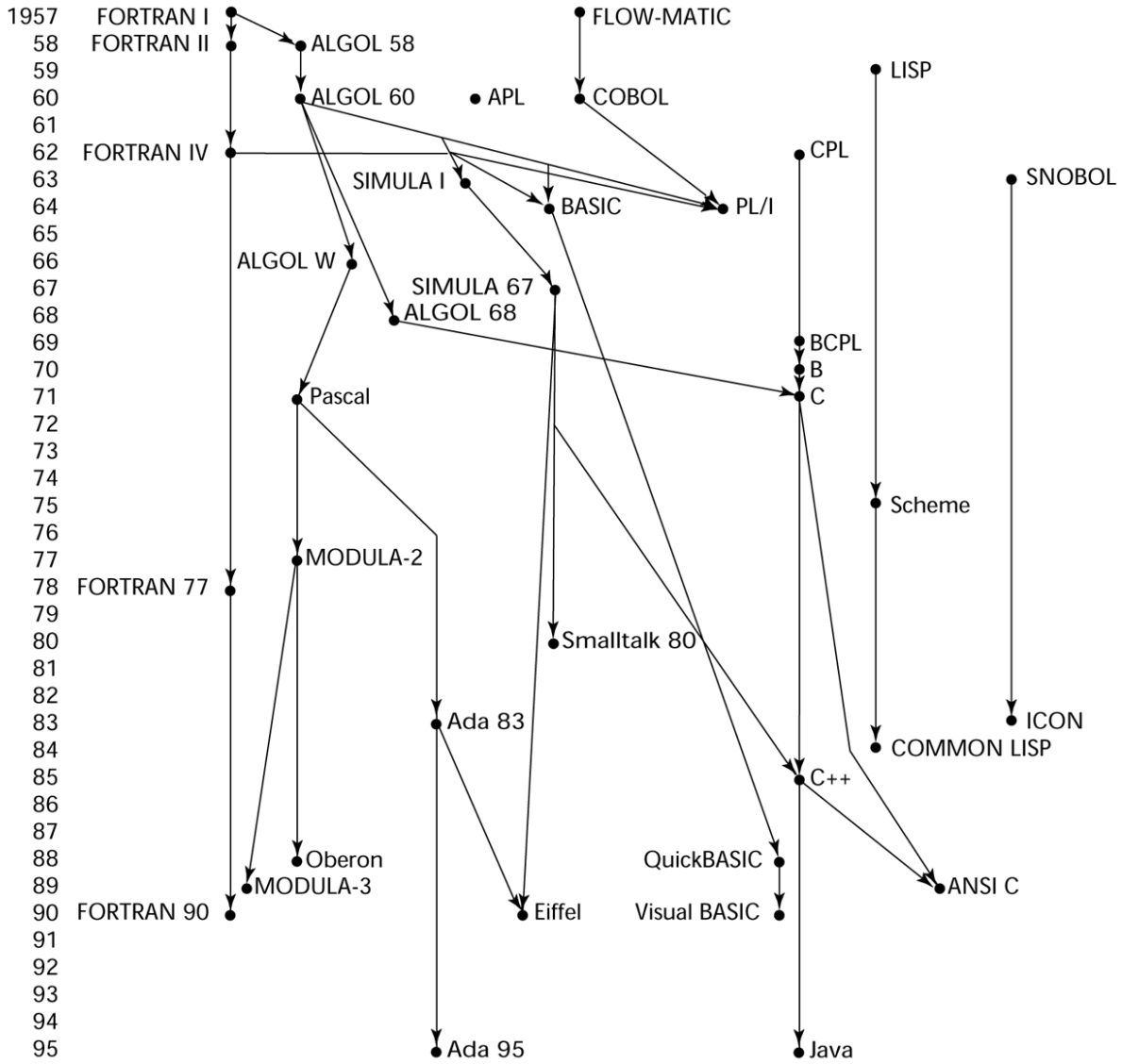
Why C?

- Operating System (OS)
- Embedded System (ES)
- Microcontroller based programming (Robotics)
- System Programming
- Programming Language Development
- Game Engine
- Programming Contest

History of Programming Language



Computer Programming

- **Computer programming** is the process of designing, writing, testing, debugging and maintaining the source code of computer programs.
- This source code is written in one or more programming languages (such as C++, C#, Java, Python, Smalltalk, etc.).

Purpose of Programming

- The purpose of programming is to create a set of instructions that computers use to perform specific operations or to exhibit desired behaviors.

Type of Programming Language

- **Machine languages**
- **Assembly languages**
- **Higher-level languages**

Machine Languages

- Machine languages (first-generation languages) are the most basic type of computer languages, consisting of strings of numbers the computer's hardware can use.

- Different types of hardware use different machine code. For example, IBM computers use different machine language than Apple computers

Assembly Languages

- Assembly languages (second-generation languages) are only somewhat easier to work with than machine languages.
- To create programs in assembly language, developers use cryptic English-like phrases to represent strings of numbers.
- The code is then translated into object code, using a translator called an assembler.

```
;CLEAR SCREEN USING BIOS
CLR: MOV AX,0600H        ;SCROLL SCREEN
     MOV BH,30           ;COLOUR
     MOV CX,0000         ;FROM
     MOV DX,184FH        ;TO 24,79
     INT 10H             ;CALL BIOS;
;INPUTTING OF A STRING
KEY: MOV AH,0AH          ;INPUT REQUEST
     LEA DX,BUFFER       ;POINT TO BUFFER WHERE STRING STORED
     INT 21H             ;CALL DOS
     RET                 ;RETURN FROM SUBROUTINE TO MAIN PROGRAM;
; DISPLAY STRING TO SCREEN
SCR: MOV AH,09           ;DISPLAY REQUEST
     LEA DX,STRING       ;POINT TO STRING
     INT 21H             ;CALL DOS
     RET                 ;RETURN FROM THIS SUBROUTINE;
```

Assembly code

Assembler

```
0001010010110101010101010101010100010
1110110101010101010101110010100010110
0010100101010010111101011101011101010
1001010010110101010101010101010110110
0110100100110010111101011101010100010
0001000101011101010101000101010111010
1010100101010010101101011101011101011
0001010010110101010101010101010100010
```

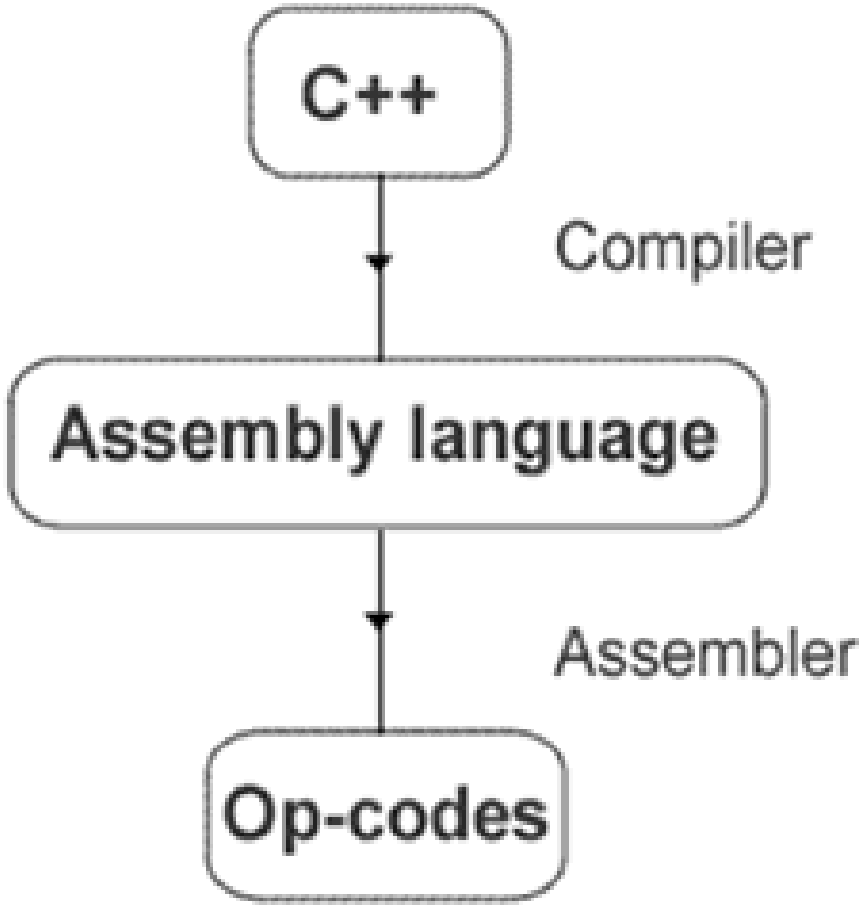
Object code

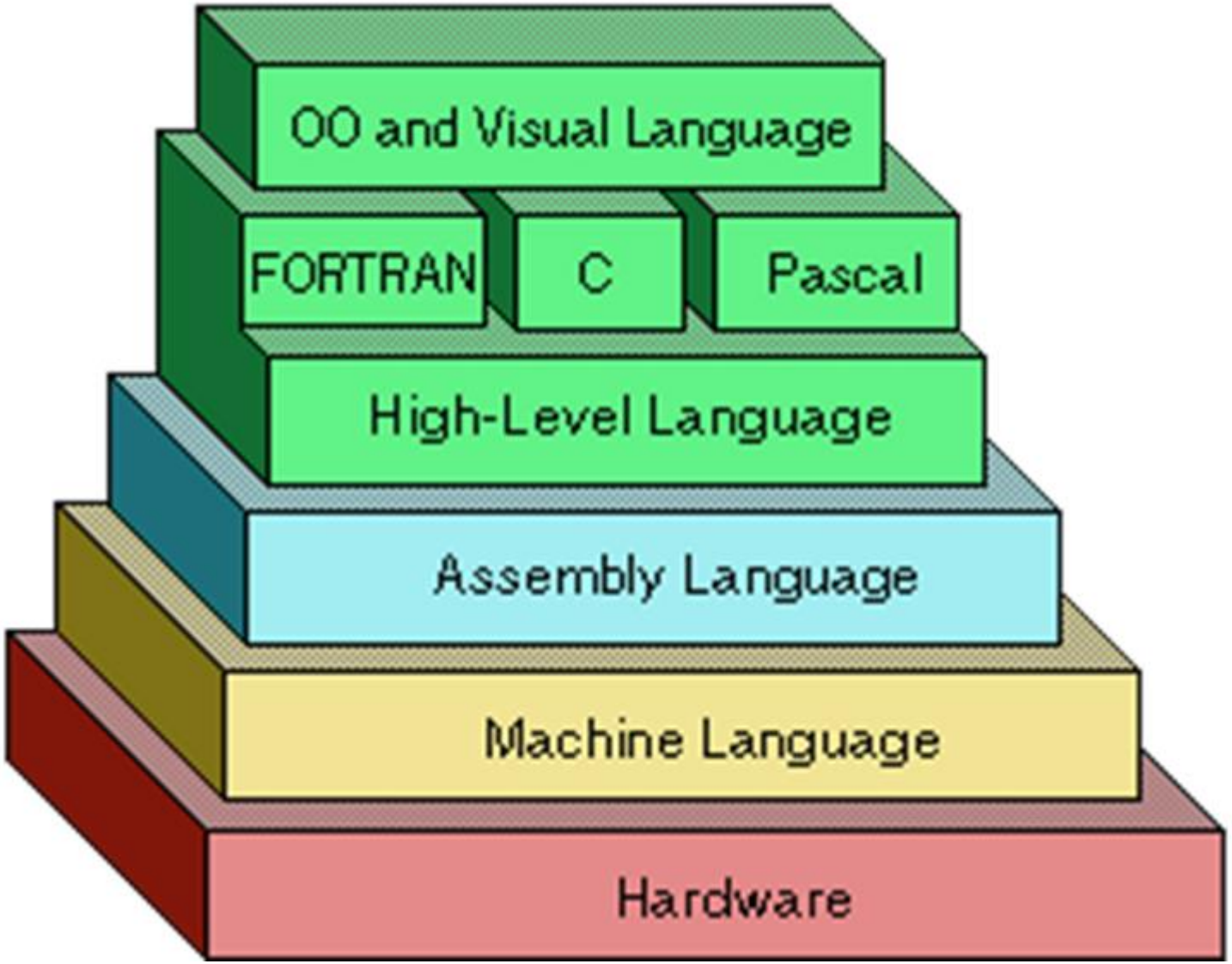
Higher-Level Languages

- Higher-level languages are more powerful than assembly language and allow the programmer to work in a more English-like environment.

- Higher-level programming languages are divided into three "generations," each more powerful than the last:

    - Third-generation languages
    - Fourth-generation languages
    - Fifth-generation languages


C++
Compiler
Assembly language
Assembler
Op-codes


OO and Visual Language
FORTRAN
C
Pascal
High-Level Language
Assembly Language
Machine Language
Hardware

Languages!

COBOL
ALGOL
FORTRAN
LISP
APL
PROLOG
PILOT
PL/1
PASCAL
MODULA 2
FORTH
MACHINE LANGUAGE
ASSEMBLE
LOGO
ADA
DOD
C
UNIX
BASIC
A⇒B
A
B
W
C
flaphuus '85

Translator

- Translators are just computer programs which accept a program written in high level or low level language and produce an equivalent machine level program as output. Translators are of three types:
    - Assembler
    - Compiler
    - Interpreter

Assembler

- Assembler is used for converting the code of low level language (assembly language) into machine level language.

Compiler

- The compiler is one kind of system software that translates the programs written in high level language to machine language.

Interpreter

- The interpreter is a system software which use to convert high level language programs to machine language. But it convert one line at a time and execute it then it convert next line and so on.

Compiler vs Interpreter

- A complier converts the high level instruction into machine language while an interpreter converts the high level instruction into an intermediate form.
- The compiler executes the entire program at a time, but the interpreter executes each and every line individually.
- List of errors is created by the compiler after the compilation process while an interpreter stops translating after the first error.
- Autonomous executable file is generated by the compiler while interpreter is compulsory for an interpreter program.
- Interpreter is smaller and simpler than compiler
- Interpreter is slower than compiler.